DiffShield v2: protecting face photos from diffusion-model editing

Project report · written 2026-10-22 · all numbers are measured and copied from the generated result tables (`results/tables/`, `results/tables/final_readout.md`) or from the run logs; nothing is estimated. Experiments that were planned but not run are marked "not run". CelebA-HQ face images were used for research only; no face image appears in this report (the before/after panels stay local).

Contents

1. The problem and the objectives
2. Threat model and scope
3. Starting point: DiffShield v1 and what was wrong with it
4. System architecture (the method, and how the code is organised)
5. Implementation methodology (what we did, step by step)
6. Evaluation design and how each metric maps to the goals
7. Results
8. Analysis
9. Did we achieve the goals?
10. Limitations
11. Conclusions and what to do next Appendix A. Reproducibility and file map · Appendix B. Problems found and fixed during validation

---

1. The problem and the objectives

1.1 The problem

People post face photos online. Anyone can give such a photo to a diffusion-model editor (Stable Diffusion image-to-image, InstructPix2Pix, inpainting, identity-conditioned editors) and ask for a changed picture: "the same person in a suit", "at a beach". The result can be a convincing fake of a real person. **DiffShield is a defensive tool:** before a photo is posted, an almost invisible pattern of noise is added to its pixels. The pattern is found by an optimiser (PGD, projected gradient descent) so that **edits made from the protected photo no longer look like the same person**, while the protected photo looks identical to the original. It protects people's own photos; it is not for generating fakes of anyone.

1.2 Objectives

#	Objective	What would count as success
G1	**Invisibility:** the protected photo looks identical to the original	PSNR ≥ 38 dB and SSIM ≥ 0.95 on **every** photo; maximum pixel change = the stated budget
G2	**Protection:** edits of the protected photo stop looking like the person	higher protection success rate (PSR) than clean photos, than a random-noise control, and than v1, with 95% confidence intervals that exclude zero

#	Objective	What would count as success
G3	**Robustness to cleanup ("purification"):** the protection survives an attacker cleaning the noise off first	protection remains after JPEG, resizing, VAE round trip, DiffPure-style denoising, restoration networks, none of which the method was trained on (held-out)
G4	**Generality:** works across editors and edit strengths	measured per editor (SD1.5, SDXL, InstructPix2Pix, inpainting, IP-Adapter FaceID) and per strength
G5	**Honest, fair evaluation:** the claims can be trusted	controls (noise), paired comparisons on the same photos, fixed train/validation/test lists, held-out cleanups, an adaptive-attacker stress test, comparison with existing tools (PhotoGuard, Mist), every number with n, CI and seeds
G6	**Reproducibility:** another person can rerun everything	one config, logged seeds/versions/git hash, tests, one command per analysis step

---

2. Threat model and scope

- **Defender.** Owns the photo and runs the protection offline, with white-box access to the Stable Diffusion 1.5 VAE (and, for some variants, the UNet). Budget: L-infinity ≤ 4/255 on the normal 0–255 pixel scale unless stated (sweep: 2, 4, 6, 8).
- **Non-adaptive attacker (the headline).** Does not know the protection. Edits with one of five editors, optionally after one of 13 cleanups. Editors: SD1.5 img2img (strengths 0.3, 0.55, 0.75), SDXL img2img, InstructPix2Pix, SD inpainting with a mask that keeps the face, IP-Adapter FaceID. Cleanups: JPEG q90/75/50 and another JPEG setting, down-up resize 0.5× and another ratio, SD VAE round trip, an SD-based "DiffPure-style" noise-then-denoise at t* = 0.05/0.1/0.2/0.3, Swin2SR restoration.
- **Adaptive attacker (stress test, reported separately).** Knows the method and trains a small autoencoder on clean/protected pairs of *other* photos, then uses it as the cleanup.
- **Not covered / not claimed:** survival in any scenario not listed; GrIDPure (model file unavailable) and IMPRESS (not run); InstantID; **human perception of identity (no human study was run)**. If an attacker masks the face itself in inpainting, those pixels are discarded before the model sees them, so noise there cannot protect them.

---

3. Starting point: DiffShield v1 and what was wrong with it

3.1 How v1 worked

PGD for 40 iterations; each step averages gradients over random transforms (Expectation over Transformation: random JPEG and crops). The loss is `α·L_vis − β·L_sem − γ·L_str`: - `L_vis`: distance between the SD1.5 VAE latent of the protected photo and of the original; **maximised** (push the editor's view of the photo away from the original). - `L_sem`: 1 − cosine similarity between the photo's CLIP embedding and the text "a potted plant"; minimised. - `L_str`: 1 − SSIM between original and protected; minimised (keeps the photo unchanged). Weights from Optuna (30 trials): α = 0.2699, β = 0.9024, γ = 60.53. Evaluated on 70 CelebA-HQ photos (PSNR 41.9 dB, SSIM 0.975).

3.2 What was wrong (and what v2 does about each)

#	Problem in v1	Consequence	v2 response
1	The budget was mis-labelled: "8/255" was measured on [-1, 1] tensors	the real budget is **4/255** on the normal scale	confirmed on real PNGs (max pixel change exactly 4 in 70/70 photos); all budgets now on the 0–255 scale
2	No random-noise control	part of the effect could be noise	uniform and PSNR-matched Gaussian noise controls

#	Problem in v1	Consequence	v2 response
3	No identity metric (LPIPS measures *change*, not *who it is*)	"disrupted" ≠ "no longer the person"	ArcFace identity similarity and PSR
4	Never tested against cleanups	an attacker can simply clean the noise off	cleanups inside the optimisation; 13 held-out cleanups in evaluation
5	One editor, one prompt, one strength, one seed, 15 photos	one lucky setting proves nothing	5 editors, several strengths/prompts, 3 seeds, 100 test photos
6	Optuna rewarded visible change, not protection	tunes the wrong thing	Optuna maximises PSR on validation photos
7	Attacks one stage (the VAE encoder), no ablations	cannot say what matters	ablations; denoiser and anti-purification terms; target-steered terms
8	Runtime string patches, unlogged seeds	not reproducible	patches became commits; one config; per-image seeds; full run logging

v1 was reproduced on the original 70 photos (PSNR 41.986 ± 0.416, SSIM 0.9754 ± 0.0054, LPIPS 0.2314 ± 0.0637; paired differences to v1's own numbers +0.075 dB, +0.0004, −0.0002, all inside the accepted tolerance of ±0.5 dB / ±0.005 / ±0.03).

---

4. System architecture

4.1 Two halves that never share components

```
HALF A: protection (offline)                          HALF B: attack-and-score harness (measures
every claim)
 photo x ──► PGD loop (40 steps, K = 4 draws/step) ──►  protected photo ──► [optional cleanup P*] ──►
editor ──► edit
              │  draw a cleanup P from a TRAINING mixture                                          │
              │  score the cleaned photo: latent loss, (denoiser loss),
ArcFace similarity to the source,
              │  (CLIP term), (anti-purification), (target term)
PSR, CLIP-I, edit-LPIPS, PSNR/SSIM
              │  visibility penalty on the PUBLISHED photo                                         P*
= HELD-OUT cleanups only
              └─ step by sign of gradient, clip to ±ε, round to 8-bit
```

Why two halves: if the attacker's cleanups are the ones the protection was trained against, the test proves nothing. Half B uses cleanups and editors the loop never saw.

4.2 The protection loop (Half A)

Maximise over ||δ||∞ ≤ ε the expectation over cleanups P and randomness of `α·L_vis(x, P(x+δ)) + η·L_den(P(x+δ)) − β·L_sem(P(x+δ)) − γ·L_str(x, x+δ)` - **Training cleanups (all differentiable):** JPEG (random quality), down-up (random scale), SD VAE round trip, one-step SD purifier (noise at a random timestep, one UNet step, decode). One down-up setting is held out for an early check. - **`L_str`** is computed on the published photo `x+δ`, not on the cleaned one, so the visibility penalty is on what is actually posted. - **Optional terms** (all switchable): denoiser error `L_den`; anti-purification (patch-DCT high-frequency and error-grows-with-timestep, after AntiPure); SDXL-VAE ensemble; identity loss `L_id` (implemented, **switched off in every preset**, never evaluated). - **Target-steered terms (added 2026-10-20, the main new idea):** `L_vis` only pushes the latent *away from the clean latent* and has no direction. The new term instead pulls the latent of the cleaned photo **toward a fixed destructive target**, the idea behind PhotoGuard and Mist, but inside the cleanup loop: `value =`

`-w·MSE(enc(P(x+δ)), z_target)` with `z_target = 0` ("zero latent") or the latent of a gray image. Variants: `v2_tgt_zero` (α = 0.27), `v2_tgt_zero_mid` (α = 0.55), `v2_tgt_zero_hi` (α = 1.0), `v2_tgt_gray`. - **Method presets in the code:** `v1`, `v2_cheap` (cleanups in the loop, no UNet term), `v2_main` (+ one-step purifier + denoiser term), `v2_antipure`, `v2_final` (Optuna-tuned, antipure on, SDXL ensemble), ablations `abl_vis_only / den_only / no_sem / no_pur`, `eps2/6/8`, and the `v2_tgt_*` family. Baselines: `noise_uniform`, `noise_gauss_psnr`, `photoguard` (official 1000 iterations), `mist` (official 100 iterations, official target image fetched at run time); both are our re-implementations checked against the official code. - **Efficiency choices:** fp16 autocast for the VAE in the loop (≈2× faster, loss trajectories matched fp32 on a check), gradient checkpointing, per-image seeds derived from a hash so runs are repeatable and resumable.

4.3 The evaluation harness (Half B)

Editors, cleanups, face scoring (insightface `buffalo_l` ArcFace), CLIP-I, LPIPS, an edit runner that resumes by row key and writes one CSV row per (photo, seed, editor, prompt, strength, cleanup, method), and an analysis layer (bootstrap intervals, PSR, paired tests, gates).

4.4 Code organisation

```
config/config.yaml + config/parts/*.yaml   every hyper-parameter, seed, path (deep-merged)
src/protect/    Half A: protectors, losses, purifiers, PGD, anti-purification, ensemble, identity
loss, registry
src/eval/       Half B: editors, cleanups (eval), faces, CLIP/LPIPS, masks, edit runner, analysis
(CIs, PSR, gates)
src/baselines/  noise controls, PhotoGuard, Mist, adaptive autoencoder
scripts/        run_protect, evaluate, tune_optuna, run_job, ingest_results, make_tables,
make_figures, readout_final,
                make_pareto_figure, finalize_all, make_human_study, analyze_human_study, demo, ...
kaggle/         jobs.yaml (every Kaggle session as a list of steps), job_runner.ipynb, build_bundle.py
data/splits/    the fixed photo lists          tests/   377 CPU tests         results/  CSV/JSON/logs
(images never committed)
```

4.5 Compute pipeline

No local GPU. The code is packaged as one zip (built from a clean git commit, hash printed in every log), uploaded as a private Kaggle dataset, and each session is the same notebook with a `JOB_IDS` setting. Sessions run on two T4 GPUs, shard the photos across them, checkpoint every few photos and can resume. About 30 sessions were run, spread over several accounts; outputs come back as `<SESSION>_outputs.zip` and are ingested locally by `scripts/finalize_all.py` (unpack → combine → tables → readout → figures).

---

5. Implementation methodology (what we did, step by step)

The original brief prescribed phases 0–8 over about ten weeks of sequential work; we compressed it to a few days by running experiments in parallel on Kaggle. The order followed the rule "measure before you build".

Phase	What was done	Result
0. Reproducible base	v1's runtime patches became commits; one config; run logging; budget check; Kaggle runner; v1 reproduced on 70 photos	budget = 4/255 confirmed; v1 reproduced within tolerance

Phase	What was done	Result
1. Evaluation harness	100 test photos (all 70 of v1's + 30 new), 20 messy, 20 validation, 20 train, 120 adaptive-attacker pool; 5 editors; ArcFace/PSR/CLIP/LPIPS; bootstrap intervals; threshold = 95th percentile of impostor similarity on clean-photo edits (E0)	reference edits and PSR threshold 0.101 (SD1.5)
2. Baselines and controls	noise controls, PhotoGuard, Mist; v1 under 13 cleanups	the control and the decision table
3. Cleanups in the loop	differentiable JPEG / down-up / VAE round trip inside PGD	`v2_cheap`
4. One-step purifier + denoiser loss	one-step SD purifier in the mixture; `L_den`	`v2_main`
5. Anti-purification terms	patch-DCT high-frequency and error-grows-with-t terms	`v2_antipure`
6. Tuning	Optuna: 16 trials on 10 validation photos (proxy objective), top 3 confirmed on 20 validation photos	tuned `v2_final`
7. Final experiments	E0–E10 as far as time allowed; ablations; ε sweep; adaptive attacker	section 7
New (2026-10-20) target-steered variants	diagnosed why v2_main ≈ v1 (no direction in `L_vis`); added the target term; **screened on the 20 validation photos only**; then ran the three declared variants on the 100 test photos	section 7.9
8. Write-up and demo	tables, figures, this report, `scripts/demo.py` (protect one photo, show edits), README	this document

Discipline we kept. One config; per-image seeds; every run writes its resolved config, git hash, library versions; fixed photo lists, never tuning on test; smoke tests on 5 photos before any long run; negative results kept; no number invented.

Scale-downs we had to make (all stated as limitations): DPM-Solver++ 30-step sampler for all editors (not v1's); 40 PGD iterations and K = 4 (no 100 iterations / K = 8); Optuna 16 trials on 10 photos with a proxy objective; GrIDPure unavailable and IMPRESS/InstantID not run; one CLIP model; a weak adaptive attacker; edit-LPIPS not computed at scale (CPU cost); no human study; the `no_sem` / `no_pur` ablations and ε = 6 not run.

---

6. Evaluation design and how each metric maps to the goals

6.1 Data and protocol

- **Photos:** CelebA-HQ. Fixed lists: **test 100** (including all 70 of v1's evaluation photos so v1 can be compared on the same photos), messy 20 (pose, blur, lighting, occlusion, pre-compression), **validation 20 (all tuning and screening)**, train 20, adaptive-attacker pool 120. v1's four calibration photos are excluded.
- **Edits:** 30-step DPM-Solver++, **3 seeds**, prompts rotated per photo (all three prompts for the headline setting), saved as JPEG q90.
- **Unit of analysis = the photo.** Seeds and prompts are averaged within a photo first; intervals are 95% bootstrap over photos; every comparison is **paired on the same photos**.
- **Two modes, never mixed:** non-adaptive (headline) and adaptive (stress test).

6.2 The metrics

Metric	What it measures	Goal	How it is used / what it can and cannot tell
Max pixel change (L_∞, 0–255 scale)	the budget actually used	G1	must equal the stated ε; confirmed = 4 for v1/v2 (a Gaussian control can exceed it by design)

Metric	What it measures	Goal	How it is used / what it can and cannot tell
PSNR / SSIM of protected vs original	how little the photo changed	**G1**	per-photo rule PSNR ≥ 38 dB, SSIM ≥ 0.95. PSNR measures pixel error, SSIM structural similarity. We report mean, minimum, and the *number of photos* breaking the rule (a mean can hide violations; this is exactly how `v2_final` slipped through tuning)
LPIPS (protected vs original)	perceptual difference of the protected photo	G1	secondary invisibility measure
ArcFace similarity (edit vs source photo)	how much the edit still matches the person (higher = same person)	**G2**	continuous measure, independent of any threshold; reported as the mean over edits where a face is found
PSR, protection success rate	share of edits with **no detected face or ArcFace similarity below the threshold** (the 95th percentile of the similarity between *different* people, measured on clean-photo edits: 0.1005 for SD1.5)	**G2, G3, G4**	the headline number. Operational definition of "no longer the same person". Counts "no face" as protected, so the no-face rate is always checked and reported next to it
Net protection = PSR(method) − PSR(clean photo) under the **same** cleanup, paired	the part of PSR caused by the protection	G2, G3	needed because cleanups (DiffPure especially) change clean photos too; raw PSR under DiffPure would otherwise look like protection
Paired difference vs v1 / vs noise control	improvement over the old method / over doing nothing special	G2, G5	the supported claim is "CI excludes 0"
PSR under 13 held-out cleanups	survival of cleanup	**G3**	plus the plan's gates (below)
PSR per editor and strength	generality	**G4**	shows where it does *not* work
No-face rate	face not found in the edit	G2 audit	checks the gain is not just "faces disappeared"
Per-photo fully-protected share	photos whose edits are protected for every seed/prompt	G2 audit	shows that protection is probabilistic per edit
CLIP-I, edit-LPIPS	secondary similarity of the edit to the source	G2 (secondary)	CLIP-I computed; edit-LPIPS only for 513 pairs (CPU cost), **not reported**
GPU-seconds per photo	cost	practicality	v1 ≈ 46 s; v2 variants ≈ 2 min per photo per T4
Gates (E3, E4, E5, E6, invisibility)	pre-set pass/fail rules from the brief	G3, G1	reported as pass/fail with numbers; **never re-defined after the fact**
Human 2AFC	whether people can see the noise / still recognise the person	G1, G2	**not run**

Why PSR (and why we do not trust it alone). PSR turns a continuous score into one understandable number, but it depends on a threshold, so we always show the mean similarity, the no-face rate, the net protection over clean photos and the confidence interval. A recognition score is also not the same as human recognition; this is why the human study (not run) matters and why the report makes no claim about human perception.

---

7. Results

*Setting unless stated: SD1.5 image-to-image, strength 0.55, the 100 test photos, 3 seeds. * = the 95% bootstrap interval excludes 0.*

7.1 Invisibility (G1): per-photo rule PSNR ≥ 38 dB and SSIM ≥ 0.95

method	n	PSNR mean (min)	SSIM mean (min)	photos < 38 dB	photos SSIM < 0.95
v1	100	42.42 (41.17)	0.9763 (0.9587)	0	0
v2_main	100	41.72 (40.34)	0.9784 (0.9707)	0	0
v2_cheap	100	41.58 (40.28)	0.9790 (0.9694)	0	0
v2_antipure	100	41.18 (38.91)	0.9700 (0.9235)	0	13
v2_final (tuned)	100	40.46 (38.76)	0.9549 (0.9197)	0	**29**
v2_tgt_zero	100	42.16 (40.58)	0.9822 (0.9728)	0	0
v2_tgt_zero_mid	100	40.91 (39.75)	0.9728 (0.9615)	0	**0**
v2_tgt_zero_hi	100	40.05 (39.14)	0.9628 (0.9459)	0	4
eps2	100	45.34 (44.86)	0.9836 (0.9628)	0	0
eps8	100	35.98 (33.33)	0.8958 (0.7724)	**98**	**96**
PhotoGuard (4/255)	100	37.15 (36.83)	0.9061 (0.8123)	**100**	**100**
Mist (4/255)	100	38.00 (37.47)	0.9229 (0.8275)	**50**	**100**
uniform noise	100	40.80 (40.72)	0.9518 (0.9257)	0	43
Gaussian noise (PSNR-matched to v1)	100	41.91 (41.89)	0.9623 (0.9337)	0	3

The Optuna search checked the limits on the *average* over its 20 photos, not per photo, which is why `v2_final` passed the search but fails per photo. PhotoGuard and Mist also change at most 4 gray levels, but they use the full budget on nearly every pixel and have no structure penalty, so they visibly lower PSNR and SSIM.

7.2 The noise control (G2, G5)

	PSR
clean photos	13.9%
uniform noise, same budget	14.9% (+1.0 over clean [−1.0, +3.1])
Gaussian noise, PSNR-matched	14.7% (+0.8 [−1.2, +2.9])
v1	24.0% (**+10.1 over clean [+7.2, +13.0]**)

v1 beats uniform noise by +9.1 points [+6.4, +12.0]. **Noise alone does nothing; the protection effect is real.**

7.3 Protection under cleanup (G3): net protection over clean photos under the same cleanup

cleanup	clean PSR	v1	v2_main	v2_cheap	v2_final	tgt_zero	tgt_zero_mid	tgt_zero_hi
none	0.139	+0.101*	+0.104*	+0.086*	+0.156*	+0.279*	**+0.358***	+0.389*
JPEG q75	0.103	+0.047*	+0.080*	+0.063*	+0.060*	+0.107*	**+0.130***	+0.163*
JPEG q50	0.130	+0.037*	+0.050*	+0.043*	+0.023	+0.050*	**+0.073***	+0.083*
down-up 0.5	0.097	+0.000	+0.057*	+0.047*	+0.047*	+0.047*	**+0.093***	+0.093*
VAE round trip	0.123	+0.097*	+0.127*	+0.090*	+0.150*	+0.310*	**+0.400***	+0.410*
DiffPure 0.1	0.133	+0.130*	+0.147*	+0.133*	+0.163*	+0.310*	**+0.383***	+0.477*
DiffPure 0.2	0.217	+0.100*	+0.117*	+0.097*	+0.133*	+0.260*	**+0.353***	+0.383*
DiffPure 0.3	0.260	+0.100*	+0.103*	+0.087*	+0.153*	+0.237*	**+0.340***	+0.403*

The effect survives held-out DiffPure at every strength; JPEG and resizing shrink it. v1 loses it completely under down-up 0.5 while the cleanup-aware variants keep part of it. **Plan gates:** E3 "v1 barely drops" by the raw-PSR rule (this rule ignores that cleanups also change clean photos, hence net protection above); **E4 fails** (the "double v1" rule fails because v1 already survives most cleanups); **E5 passes** (v2_main PSR under DiffPure 0.1 is 0.280 against half of 0.243); E6 "go" on 1 of 3 cleanups; invisibility fails for `v2_final`.

7.4 v2 versus v1, paired (no cleanup and selected cleanups)

v2_main − v1: +0.3 points [−1.8, +2.6] without cleanup (no gain); supported only under down-up 0.5 (+5.7 [+3.0, +8.7]) and down-up (other) (+3.0 [+0.3, +6.3]). v2_final − v1: **+5.4 [+2.4, +8.7]** without cleanup, +4.7 under down-up 0.5, +5.3 under the VAE round trip, +5.3 [+0.3, +10.7] under DiffPure 0.3, within noise elsewhere (but v2_final breaks the invisibility rule).

7.5 What each loss term contributes (E7, ablations; n = 100, no cleanup)

`vis_only` 25.3% [19.7, 31.0] (+1.3 vs v1, not significant); `den_only` 12.3% [8.0, 17.0] (−11.7 vs v1; its optimisation left an almost invisible perturbation, PSNR 50.8 dB, so this is a weak test of the denoiser idea); v2_main ("both") 24.3%. In our implementation **the encoder-shift term does the work**. `v2_antipure − v1`: +4.7 [0.0, +8.7] under the VAE round trip and +5.0 [0.0, +10.0] under DiffPure 0.1 (borderline), with 13/100 photos below SSIM 0.95. Not run: the `no_sem` and `no_pur` ablations.

7.6 Budget (ε sweep, E9) and the adaptive attacker (E8)

ε	method	PSNR	SSIM	PSR none	PSR JPEG 75	PSR DiffPure 0.1	PSR FaceID
2/255	eps2	45.34	0.984	0.17 [0.12, 0.23]	0.17	0.20	0.00
4/255	v2_final	40.46	0.955	0.29 [0.24, 0.35]	0.16	0.30	0.00
8/255	eps8	35.98	0.896	0.38 [0.31, 0.45]	0.21	0.40	0.00

Budget is the main lever, and JPEG flattens the gain at every budget. ε = 6 was not run.

Adaptive attacker (autoencoder purifier, 120 other photos, 35,496 steps, output PSNR vs the clean photo 45.1 dB): PSR under it is 9.3% for clean photos, **11.0% for v1** (net +1.7 [−1.0, +4.7]) and **10.7% for v2_final** (net +1.3 [−1.0, +3.7]): **the protection is removed**. The attacker is weak by construction, so this is a lower bound on the damage. It was **not** run against the target-steered variants.

7.7 Other editors and strengths (G4), no cleanup

method	SD1.5 s0.3	SD1.5 s0.75	FaceID	InstructPix2Pix	inpainting	SDXL
clean	0.000	0.840	0.000	0.180	0.000	0.013
v1	0.000	0.853	0.000	0.090 (−0.090*)	0.000	0.020
v2_final	0.000	0.873	0.003	0.033 (−0.147*)	0.000	0.020
v2_tgt_zero_mid	0.000	0.900 (+0.060*)	0.020 (+0.020*)	not run	not run	not run
PhotoGuard	0.170 (+0.170*)	0.957 (+0.117*)	0.027 (+0.027*)	*0.717 (+0.537)*	0.000	0.027
Mist	0.047 (+0.047*)	0.940 (+0.100*)	0.037 (+0.037*)	0.043 (−0.137*)	0.000	not run

Nothing meaningfully protects the identity-conditioned FaceID editor, the inpainting editor (the face is masked out, so noise cannot reach it) or SDXL (a different VAE), and strength 0.3 (the edit keeps almost everything). At strength 0.75 clean edits already destroy identity (84%), so gains cannot be measured there. InstructPix2Pix is a notable exception for PhotoGuard (71.7%) and an anomaly for v1/v2_final, whose protected edits count as *less* protected than clean ones (partly a face-detection effect that we did not separate).

7.8 Comparison with existing tools (E2), 100 test photos, run at our budget 4/255

	PSR none	JPEG 75	JPEG 50	down-up 0.5	VAE round trip	DiffPure 0.1	per-photo invisibility
PhotoGuard	**0.810**	0.183	0.167	0.137	0.590	0.673	fails on 100/100 photos
Mist	0.631	0.210	0.157	0.127	0.590	0.640	fails on 100/100 (SSIM), 50/100 (PSNR)
v2_tgt_zero_mid	0.497	**0.233**	**0.203**	**0.190**	0.523	0.517	passes on 100/100

PhotoGuard and Mist protect more without cleanup, but only by visibly altering the photo. Under JPEG and resizing, `v2_tgt_zero_mid` is **ahead of PhotoGuard** (paired difference JPEG 75 +5.0 [+1.3, +9.0]; JPEG 50 +3.7 [+0.3, +7.0]; down-up 0.5 +5.3 [+2.7, +8.3]) and ahead of Mist under JPEG 50 (+4.7) and down-up (+6.3); under the VAE round trip and DiffPure 0.1 the visible tools stay ahead (mid minus PhotoGuard −6.7 and −15.7). Caveat: our re-implementations at our budget; at their own larger budgets they would be stronger and more visible.

7.9 The target-steered variants (the main new result)

Screening on the 20 validation photos (clean 8.3%, v1 15.0%; the control `v2_cheap` has the same cleanups without the target term): | method | PSNR (min) | photos failing the rule | PSR none | vs v1 | PSR JPEG 75 | PSR DiffPure 0.1 | |---|---|---|---|---|---|---| | v2_cheap (control) | 41.65 (40.62) | 0 | 11.7% | −3.3 | 13.3% | 16.7% | | v2_tgt_zero | 42.07 (40.62) | 0 | 35.0% | **+20.0*** | 13.3% | 41.7% | | v2_tgt_zero_hi | 39.99 (39.16) | 1 of 20 | 46.7% | **+31.7*** | 26.7% | 58.3% | | v2_tgt_gray | 41.94 (40.38) | 0 | 35.0% | +20.0* | 15.0% | 28.3% | The control does not gain, so the gain comes from the target term.

Test photos (n = 100), PSR with the paired difference to v1: | method | none | JPEG 75 | down-up 0.5 | VAE round trip | DiffPure 0.1 | DiffPure 0.3 | photos failing the invisibility rule | |---|---|---|---|---|---|---|---| | clean | 0.139 | 0.103 | 0.097 | 0.123 | 0.133 | 0.260 | n/a | | v1 | 0.240 | 0.150 | 0.097 | 0.220 | 0.263 | 0.360 | 0 | | v2_tgt_zero | 0.418 (+0.178 [+0.141, +0.217]*)* | *0.210 (+0.060*) | 0.143 (+0.047*)* | *0.433 (+0.213*) | 0.443 (+0.180*)* | *0.497 (+0.137*) | 0 | | **v2_tgt_zero_mid** | *0.497 (+0.257 [+0.218, +0.298])* | *0.233 (+0.083*) | 0.190 (+0.093*)* | *0.523 (+0.303*) | 0.517 (+0.253*)* | *0.600 (+0.240*) | **0** | | v2_tgt_zero_hi | 0.528 (+0.288 [+0.238, +0.338]*)* | *0.267 (+0.117*) | 0.190 (+0.093*)* | *0.533 (+0.313*) | 0.610 (+0.347*)* | *0.663 (+0.303*) | 4 |

`v2_tgt_zero_mid` roughly **doubles v1** without cleanup (0.497 vs 0.240), under down-up (0.190 vs 0.097) and under DiffPure 0.1 (0.517 vs 0.263); about 2.4× under the VAE round trip; 1.6× under JPEG 75 and 1.2× under JPEG 50 (the weakest point). Protection grows with the target weight (zero < mid < hi), a clean dose-response. The weights were never tuned on the test photos, but *naming* `mid` as the best variant that keeps every photo invisible was read from the test table (post hoc, among three variants declared in advance; all three are shown). The adaptive attacker was not run against these variants.

7.10 Audit: does PSR really measure what we claim?

- **Not an artefact of missing faces.** No-face edits: 0.3% (clean), 0.3% (v1), 1.4–2.6% (target variants), 8.4% (PhotoGuard), 8.2% (Mist). With those edits removed, PSR is 13.6% / 23.7% / 40.9% / 48.6% / 51.5% / 79.2% / 59.8% (clean, v1, tgt_zero, tgt_zero_mid, tgt_zero_hi, PhotoGuard, Mist).
- **The similarity itself falls smoothly:** mean ArcFace similarity of edits (face found) 0.222 (clean) → 0.177 (v1) → 0.158 (v2_final) → 0.129 / 0.109 / 0.098 (target zero / mid / hi) → 0.085 (Mist) → 0.045 (PhotoGuard).
- **Per photo the protection is probabilistic:** the share of photos protected for *every* seed and prompt is 0.0% (clean), 2.0% (v1), 5.0% (`mid`), 35.0% (PhotoGuard).

- **How it protects (visual check of the edits):** the target-steered variants and PhotoGuard protect by **degrading or restyling the edit** (cartoon, sketch or sepia looks, texture artifacts, sometimes a different-looking face), not by producing a believable different person; v1's edits look almost like the clean ones; some protected edits are still recognisable to a person.
- Even clean edits at strength 0.55 already lower ArcFace similarity a lot (mean 0.222), so the 0.101 threshold is a stringent bar and every number is relative to clean edits. Many cells are compared; the intervals are not corrected for multiple comparisons.

Figures (no faces): `results/figures/pareto_psr_vs_psnr.png` (headline: protection against invisibility for every method), `psr_vs_psnr.png` (ε sweep), `psr_by_purifier.png`, `per_editor.png`, `ablation.png`, `sim_distributions.png`, `optuna_history.png`.

---

8. Analysis

1. The protection effect is real, modest for v1, and bounded by budget. Random noise at the same budget does nothing (14.9% against 13.9% for clean photos), while v1 gains about 10 points over clean photos. The effect depends strongly on the budget (17% → 24–29% → 38% at ε = 2, 4, 8) and on the edit strength: nothing at strength 0.3, and saturation at 0.75.

2. The original v2 idea (training against cleanups, plus extra loss terms) did not beat v1 by much. `v2_main` equals v1 without cleanup, and its only supported advantage is under resizing. The ablations say why: the encoder-shift term (`L_vis`) carries the whole effect; the denoiser-only term, as implemented, produced almost no perturbation; anti-purification gave borderline gains at a visibility cost. The tuned `v2_final` is better than v1 (+5.4) but breaks the invisibility rule on 29% of photos, because tuning checked averages rather than every photo.

3. The missing ingredient was direction. `L_vis` pushes the latent away from the clean one in no particular direction. Pulling it toward a fixed destructive target (the PhotoGuard/Mist mechanism) inside the cleanup loop roughly doubled protection while keeping every photo invisible (`mid`: 49.7% PSR, 0/100 violations). The control run (same cleanups, no target term) did not gain, which isolates the target term as the cause. *Mechanism (hypothesis, not tested here):* a target latent gives the optimiser a consistent objective, so the attack forces the editor's output toward degraded content instead of merely shifting its starting point.

4. Why the existing tools score higher and why that is not the whole story. PhotoGuard (81%) and Mist (63%) win on raw PSR without cleanup, but they pay in visibility: every test photo breaks the invisibility rule (PSNR 37.2 dB, SSIM 0.906 for PhotoGuard). They also lose most of their gain under JPEG and resizing (PhotoGuard 18% under JPEG 75), where `v2_tgt_zero_mid` (23%) is ahead at much lower visibility. So the honest picture is a **trade-off between protection, invisibility and robustness to cleanup**, not a single ranking; the headline figure shows where each method sits.

5. Cleanups. DiffPure-style denoising does not remove the effect (net +10 to +48 points depending on the method), which is plausible because a diffusion-based cleanup removes noise of a different kind than a targeted latent attack. JPEG (a low-pass filter that deletes high-frequency perturbation) and down-up resizing are the effective cleanups: they remove most of v1's effect, and the cleanup-aware variants recover part of it (still the weakest spot at JPEG 50).

6. Where it does not work, and why (hypotheses). The protection attacks the SD1.5 VAE encoder, which image-to-image editors use. It does little for IP-Adapter FaceID (the identity comes from a face-embedding

network on the photo, not from the VAE path), inpainting (the face is masked, so pixels there never reach the model) or SDXL (a different VAE, which our protection did not target in the final variants). These are consistent with the design but were not individually tested.

7. The adaptive attacker is the main caveat. A weak autoencoder trained on 120 other photos brings v1 and `v2_final` back to the clean level (11% vs 9%). We did not test the target-steered variants against it; the result suggests we should not assume they survive.

8. How to read "protected". PSR is an operational, ArcFace-based definition. The audit shows it is not driven by missing faces and that the similarity itself falls, but the protected edits are often *degraded or stylised* rather than changed into a different believable person, and some stay recognisable to a person. No human study was run, so no claim about human perception is made.

9. Method lessons. Averages hide per-photo violations (invisibility); raw PSR under a cleanup can mislead, so protection should be reported net of the clean baseline; a control run isolates a cause; and screening on validation photos before spending test-set compute kept the final comparison honest.

---

9. Did we achieve the goals?

Goal	Outcome	Verdict
G1 invisibility	v1, v2_main, v2_cheap, `tgt_zero` and `tgt_zero_mid` pass the per-photo rule on all 100 photos; `v2_final`, antipure, ε = 8, PhotoGuard, Mist do not	**achieved** for the recommended variants
G2 protection	clearly above clean and noise; v1 +10 points, **`tgt_zero_mid` +36 points (about 2× v1)**, all with intervals excluding 0, for SD1.5 image-to-image at moderate strength	**achieved in one regime; modest elsewhere**
G3 robustness to cleanup	survives DiffPure and the VAE round trip; reduced by JPEG and resizing; removed by an adaptive attacker (tested on v1 and `v2_final`)	**partial**
G4 generality	does not protect FaceID, inpainting, SDXL or strength 0.3	**not achieved**
G5 fair evaluation	controls, pairing, fixed splits, held-out cleanups, adaptive test, existing-tool comparison, intervals everywhere	**achieved**
G6 reproducibility	one config, per-image seeds, logged environments, 377 tests, hash-stamped bundles, one command per analysis step	**achieved**

One-sentence verdict: v2 as originally designed (cleanups and extra terms) added little over v1; adding a **target-steered term** produced a measurable, supported improvement (about twice v1's protection at invisible levels for SD1.5 image-to-image editing), with clear limits (JPEG, adaptive attackers, FaceID/inpainting/SDXL, human perception untested).

---

10. Limitations (all real)

DPM-Solver++ sampler for every editor (not v1's); the DiffPure-style cleanup is SD1.5-based, not the original DiffPure; GrIDPure unavailable, IMPRESS and InstantID not run; 40 PGD iterations, K = 4; Optuna on 10 photos with a proxy objective, where the proxy's best trial (0.10) fell to 0.05 on the 20-photo confirm and tuned vs hand-set is not separable (0.067 vs 0.042); identity loss implemented but never evaluated; one CLIP model; de-duplication of identities used an unverified similarity threshold and the splitter's face detector missed 38 of 100 test faces (evaluation now pads the image; the test list was not re-de-duplicated); adaptive attacker weak and not run on the target-steered variants; choice of "best" target variant is post hoc among three declared variants; PhotoGuard and Mist are our re-implementations run at our budget; edit-LPIPS not reported; **no**

human study; the `no_sem` and `no_pur` ablations and ε = 6 not run; v1's Phase 2 outputs were lost (its LPIPS 0.46 can only be quoted); many comparisons without multiplicity correction; results are for CelebA-HQ portraits only.

---

11. Conclusions and what to do next

Conclusions. (i) v1's claims were weaker than stated and its budget was mis-labelled (4/255, not 8/255). (ii) A fair evaluation shows the protection effect is real, bounded by budget and edit strength, and vulnerable to JPEG, resizing and adaptive attackers. (iii) Training against cleanups alone did not beat v1 much. (iv) A target-steered term inside the cleanup loop about doubles protection while keeping every test photo invisible, and keeps more of its protection than visible tools under JPEG and resizing.

Next steps, in order of value. 1. Run the **adaptive attacker** against `v2_tgt_zero_mid` (the most important unknown). 2. A **human study** (is the noise visible; is the person still recognisable in the degraded edits). 3. Fix the weak spots: JPEG robustness (stronger cleanup mixture), SDXL/ensemble targets, and a face-embedding term for FaceID (the implemented identity loss, evaluated with a second recogniser to avoid grading on the training network). 4. Re-tune with **per-photo** invisibility constraints and more iterations/draws; run `no_sem`, `no_pur` and ε = 6. 5. Replace the post-hoc choice of `mid` by a rule fixed on validation photos.

Appendix A. Reproducibility and file map

- Config: `config/config.yaml + config/parts/*.yaml` (the tuned `final.yaml` came from Optuna trial 5; its trials are in `reports/optuna_trials_S06.csv`).
- Bundles: every Kaggle log prints the git hash of the code it ran. Hashes used: `9e1128d` (S07–S09), `64b0815` (S06), `b72e5ff` (most later sessions), `801245d` (screening S24/S25), `0b36407` (S26–S28).
- Analysis chain: `python scripts/finalize_all.py` (unpack new zips → `ingest_results.py` → `make_tables.py` → `readout_final.py` → `make_figures.py`), then `python scripts/make_pareto_figure.py`.
- Tests: `python -m pytest -q tests` (377 pass, 8 skipped; CPU only, stand-in models).
- Demo: `python scripts/demo.py --image <photo> --out demo_out` (needs a GPU for the real models; `--cpu-dry-run` checks the wiring only).
- Detailed plans and earlier reports: `docs/EVAL_PLAN_FINAL.md`, `docs/COST_MEASURED.md`, `reports/PHASE_0.md`, `reports/REPORT_v2_draft.md`.
- Data hygiene: results CSV/JSON/logs are committed; protected and edited images (face-derived) are never committed, but 84 early smoke-test JPEGs remain inside older git commits (nothing has been pushed; they must be removed from history before any push).

Appendix B. Problems found and fixed during validation (relevant to trust in the numbers)

1. Baselines did not tell the runner which GPU they used (crash). 2. CLIP scoring crashed under the newer `transformers`. 3. One missing model file (GrIDPure) aborted a whole job; now skipped and logged. 4. **Face detection missed 38 of 100 test photos** (tight crops); fixed by padding. 5. The GPU build of `onnxruntime` needed CUDA 13; pinned a CUDA-12 build. 6. A race while unpacking the face model gave garbage embeddings; now locked and checked. 7. FaceID could not load because of an old preinstalled library. 8. One failing editor aborted the others; failures are now isolated. 9. `--limit N` picked different

photos in protection and evaluation; both now sort identically. 10. Two Optuna workers raced to create the database. 11. **The purifier-tier evaluation jobs silently expanded to zero jobs** (a default of `--purifier none`), which would have left the main robustness experiments empty while the sessions still reported success; fixed and covered by a test. 12. The result combiner had been mixing smoke-test rows into the real results; now excluded. 13. Several launches used a wrong input path or the CPU instead of the GPU and were caught from the logs.